Decision Tree Classifier

Data ko sawal pooch-pooch kar tod kar decision tak pahunchne wala algorithm — aur ye kaise decide karta hai ki kaun sa sawal pehle poochna hai.

1. Introduction — Decision Tree Kya Hai?

Decision Tree ek aisa algorithm hai jo data ko **baar-baar sawal** (if-else conditions) pooch kar chhote-chhote hisso me baant deta hai, aur aakhir me ek **decision (prediction)** deta hai. Ye ulte ped jaisa dikhta hai: upar **root node**, beech me **decision nodes**, aur sabse neeche **leaf nodes** jahan final answer milta hai. Teacher ne notes me iske do major types likhe hain: **ID3** aur **CART** (CART par tick lagaya hai).

1.1 If-Else Se Samjho

Teacher ka example: ek person ki **age** dekh kar batana ki wo school me hai, college me hai ya pass out ho chuka hai. Ye bilkul normal if-else code hai:

```
age = 14
if (age <= 15):
    print("The person is in School")
elif (age > 15 and age <= 21):
    print("The person may be College")
else:
    print("The person has passed")
```

Isi code ko tree ki tarah dekho: root node par sawal hai **age ≤ 15?** Agar **Yes** to seedha leaf: "person is in school". Agar **No** to agla sawal **age > 15 and age ≤ 21?** — uske Yes/No se aage ke leaf nodes mil jaate hain. Age = 14 ke liye: 14 ≤ 15 sach hai, isliye answer "School".

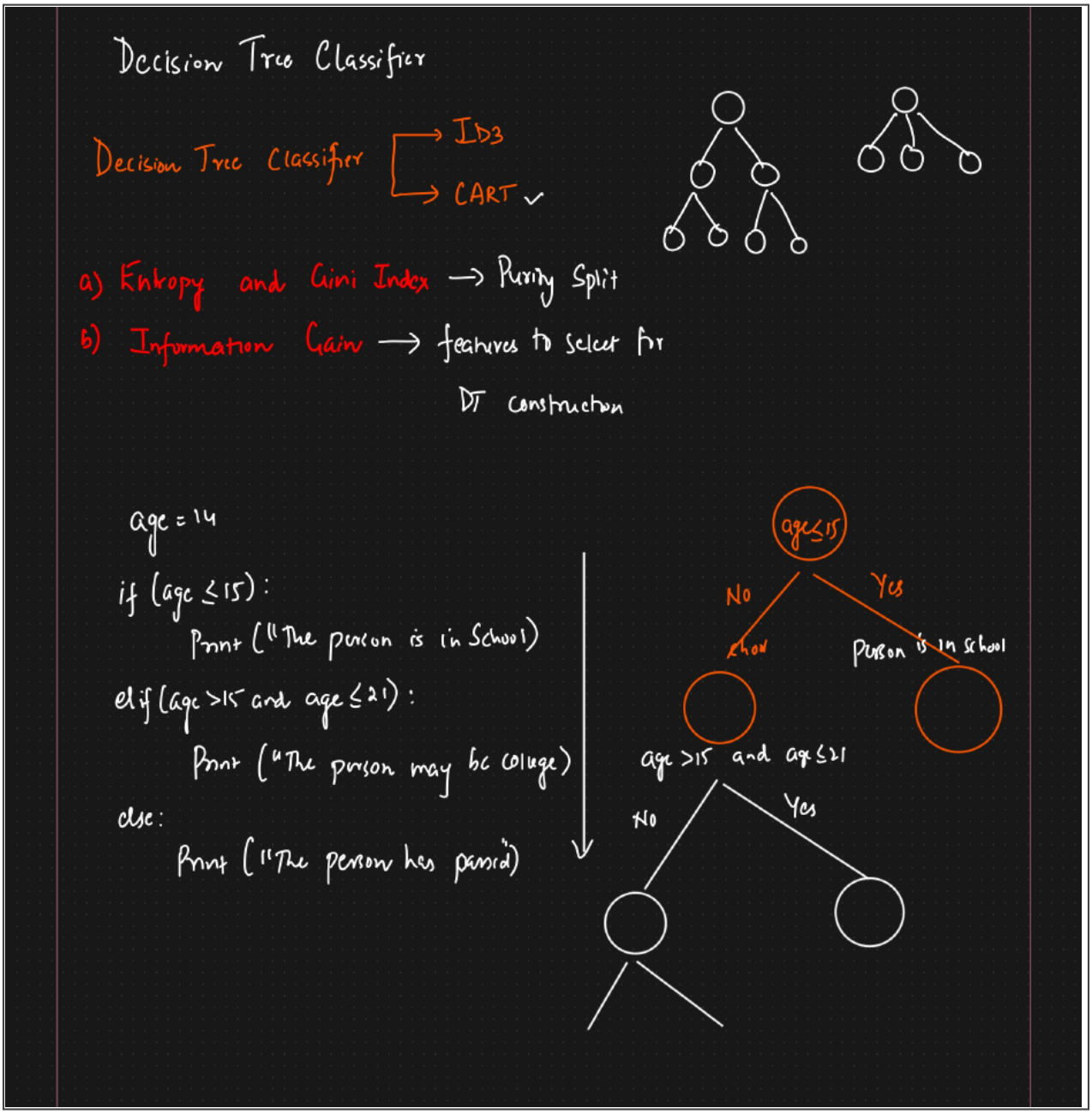
Term	Matlab
Root Node	Sabse upar ka node, jahan poora dataset hota hai aur pehla sawal poocha jaata hai
Decision (Internal) Node	Beech ke nodes jahan koi feature par condition lagti hai aur data aage tootta hai
Branch	Condition ka jawab (Yes / No ya Sunny / Overcast / Rain)
Leaf Node	Aakhri node jahan se aur split nahi hota; yahi final prediction deta hai

> **Dhyaan dein:** Decision Tree asal me **nested if-else** hi hai. Farak ye hai ki yahan humein khud nahi likhna padta — algorithm data se seekhta hai ki **kaun sa feature pehle** aur **kis threshold par** sawal poochna hai. Isi kaam ke liye do cheezein chahiye jo teacher ne top par likhi hain:

- **(a) Entropy aur Gini Index** → batate hain ki ek split **pure** hai ya **impure** (Purity Split).
- **(b) Information Gain** → batata hai ki tree banane ke liye **kaun sa feature** select karna hai.

Point (extra table)	ID3	CART
Split criteria	Entropy + Information Gain	Gini Impurity (sklearn me default)
Branches	Ek node se kai branches ban sakti hain (jaise Outlook → 3)	Hamesha 2 branches (binary tree)
Use	Sirf classification	Classification aur Regression dono

Original Handwritten Notes — Decision Tree, Page 1



Isme dikhaya gaya hai: Decision Tree Classifier ke types (ID3, CART), do key concepts (Entropy/Gini Index aur Information Gain), age wale if-else code ka example aur uska decision tree me conversion.

2. Dataset Aur Problem — Pure aur Impure Split

Ab ek asli dataset lete hain — wahi **Play Tennis** dataset jo Naive Bayes me tha. Ye **binary classification** hai (output Yes ya No). Teacher ne Outlook, Temperature, Humidity aur Wind (4 input features) par tick lagaya hai.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Poore dataset (root node) me 14 rows hain: **9 Yes aur 5 No**, yaani teacher ke shabdon me **9Y | 5N**. Maan lo hum pehle **Outlook** par split karte hain. Teen branches banti hain, aur har branch me Yes/No ki ginti ye hai:

Branch	Yes | No	Split kaisa hai?	Aage kya?
Sunny	2Y | 3N	Impure (dono class mixed)	Aur split karna padega
Overcast	4Y | 0N	**Pure** (sirf Yes)	**Leaf node** — yahin ruk jao, answer = Yes
Rain	3Y | 2N	Impure	Aur split karna padega

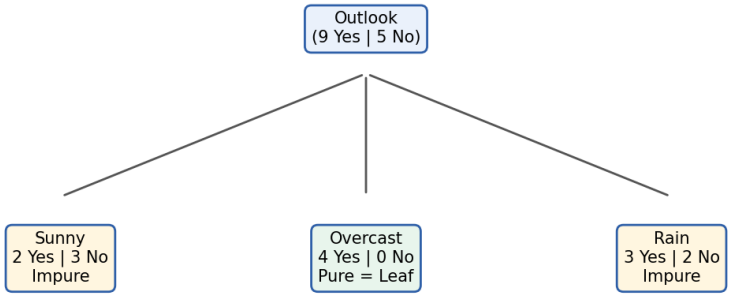


Figure: Outlook par split karne se bana tree (teacher ke notes ka diagram).

Pure split matlab node me sirf ek hi class ke samples hain (jaise 4Y | 0N). **Impure split** matlab dono class mili hui hain (jaise 2Y | 3N). Pure node par kaam khatam — wo leaf ban jaata hai. Impure node ko aur todna padta hai. Yahan do sawal khade hote hain, jinke jawab teacher ne notes me diye hain:

- **Sawal 1:** Kaise pata chale ki split pure hai ya impure? → **Entropy** aur **Gini Impurity**.
- **Sawal 2:** Splitting ke liye kaun sa feature select karein? → **Information Gain**.

3. Entropy — Purity Naapne Ka Pehla Tareeka

Binary classification me **Entropy** ka formula:

$$H(S) = -\,p_{+}\,\log_2 p_{+} - p_{-}\,\log_2 p_{-}$$

Yahan $\mathbf{p_+}$ = node me Yes ka fraction aur $\mathbf{p_-}$ = No ka fraction hai. Entropy ek tarah se node ki "ashudhi / uncertainty" naapti hai: jitni zyada mixing, utni zyada entropy.

Teacher ka example: ek node jisme **6 Yes | 3 No** hain, feature f_1 par split hoke do child nodes banate hain: $\mathbf{C_1}$ **= 3Y | 3N** aur $\mathbf{C_2}$ **= 3Y | 0N**.

Child C_1 (3Y | 3N):

$$H(C_1) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = -0.5 \times (-1) - 0.5 \times (-1) = 1 \rightarrow \textbf{Impure Split}$$

Child C_2 (3Y | 0N):

$$H(C_2) = -(3/3)\log_2(3/3) - 0 \times \log_2(0) = -1 \times \log_2(1) = 0 \rightarrow \textbf{Pure Split}$$

> **Yaad rakhein:** Binary classification me **Entropy = 0** matlab bilkul pure node, aur **Entropy = 1** matlab sabse zyada impure (50-50 mixing). Convention: $0 \times \log_2(0)$ ko 0 maante hain, aur $\log_2(1) = 0$ hota hai.

Kuch aur values (notes ke baaki examples se, comparison ke liye):

Node (Yes | No)	Entropy	Matlab
4Y | 0N	0	Pure
6Y | 2N	0.81	Thoda impure
9Y | 5N	0.94	Kaafi impure
3Y | 3N	1	Sabse zyada impure

4. Gini Impurity — Purity Naapne Ka Doosra Tareeka

Gini Impurity ka formula (n = classes ki sankhya, p_i = i-th class ka fraction):

$$G.I = 1 - \Sigma\,(p_i)^2$$

Binary case me ye ban jaata hai: **$G.I = 1 - (p_+^{\,2} + p_-^{\,2})$**. Wahi dono child nodes lete hain:

Node	Calculation	Gini	Matlab
C_1 (3Y | 3N)	$1 - [\,(1/2)^2 + (1/2)^2\,] = 1 - (1/4 + 1/4)$	**0.5**	Impure Split
C_2 (3Y | 0N)	$1 - (3/3)^2 = 1 - 1$	**0**	Pure Split

Teacher ne notes me Entropy aur Gini ka ek graph bhi banaya hai (x-axis par p_+ / p_-, y-axis par impurity). Dono curves 0 aur 1 par zero hote hain (pure node) aur p = 0.5 par sabse upar jaate hain:

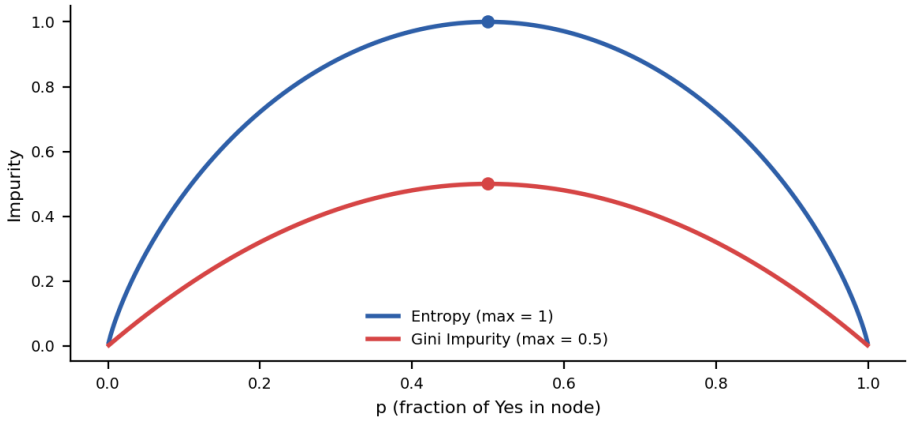


Figure: Entropy ka maximum 1 hai aur Gini ka maximum 0.5 (binary classification me), dono p = 0.5 par.

p (Yes ka fraction)	0	0.25	0.5	0.75	1
Entropy	0	0.81	1	0.81	0
Gini Impurity	0	0.375	0.5	0.375	0

Dhyaan dein: Dono ka kaam ek hi hai (purity naapna) aur dono ka nateeja similar hota hai: **0 = pure**, value jitni badi utna impure. Bas range alag hai — Entropy 0 se 1, Gini 0 se 0.5.

5. Information Gain — Kaun Sa Feature Pehle Chunein?

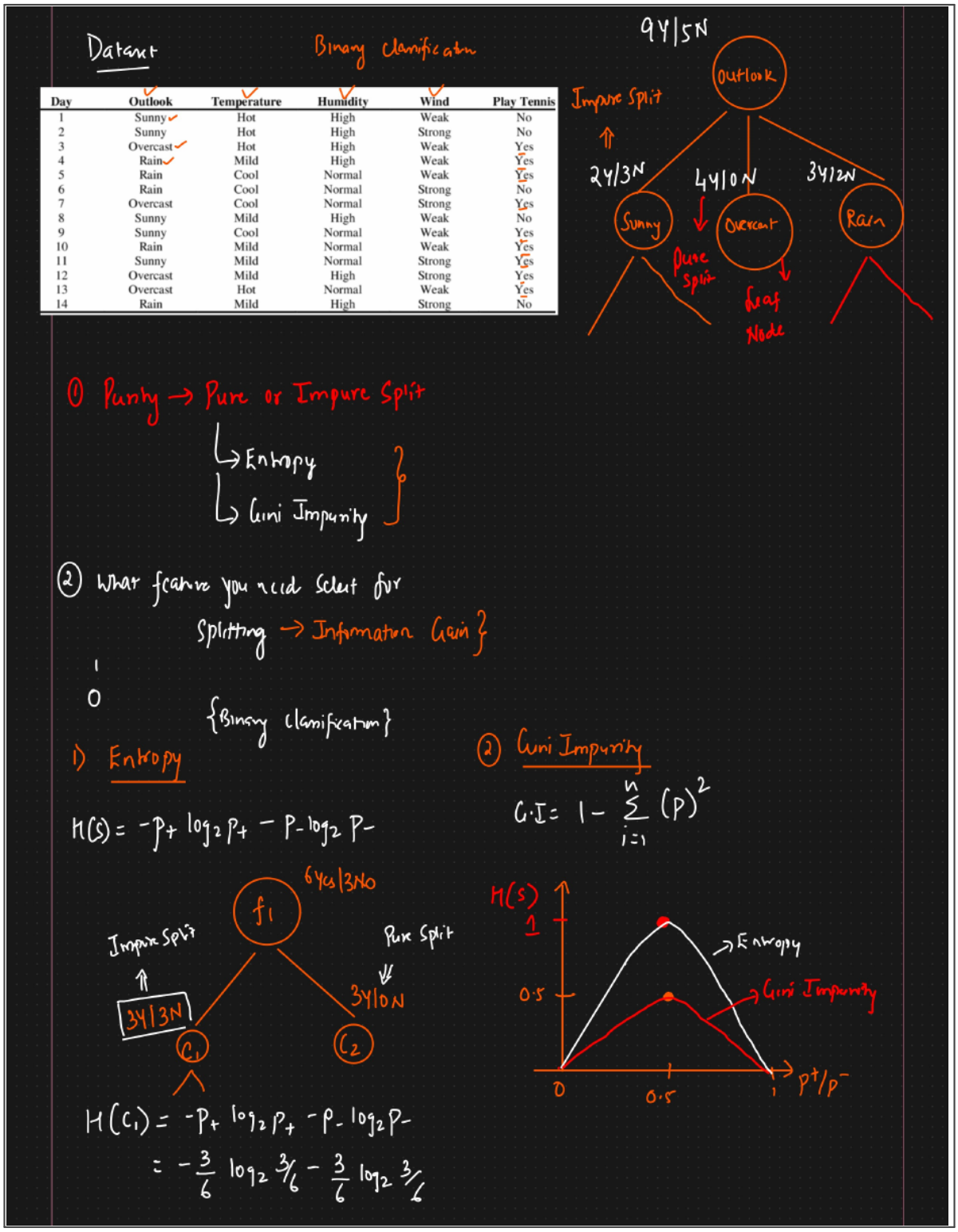
Maan lo dataset me teen features **f_1, f_2, f_3** hain. Tree me kisi ko bhi root bana sakte hain — f_1 root, ya f_3 root, ya f_2. Har choice se alag tree banta hai (teacher ne notes me alag-alag trees banaye hain). To **best feature kaise chunein?** Isi ka jawab hai **Information Gain**.

Information Gain batata hai ki kisi feature par split karne se **entropy kitni kam hui**. Jis feature se entropy sabse zyada ghate, wahi best hai:

$$\mathbf{Gain(S, f) = H(S) - \Sigma_{v \quad Val(f)} \left(|S_v| / |S| \right) \times H(S_v)}$$

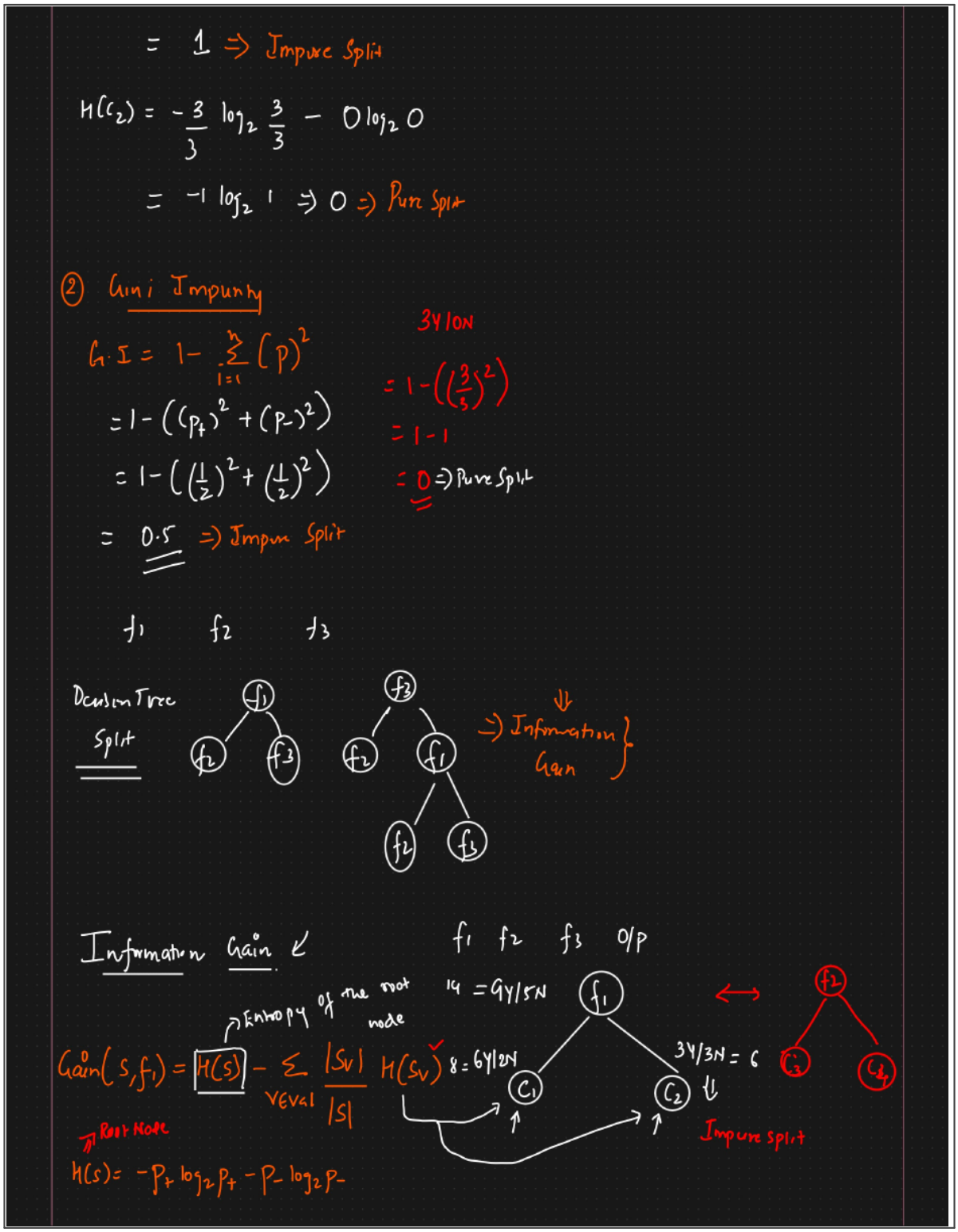
Symbol	Matlab
H(S)	Parent (root) node ki entropy — jisme poora dataset hai
S_v	Feature f ki value v wale rows (split ke baad ka child node)
$\|S_v\| / \|S\|$	Child node ka weight (child me kitne rows / parent me kitne rows)
$H(S_v)$	Us child node ki entropy

Yaad rakhein: Information Gain = Parent ki entropy − Children ki weighted average entropy. Jis feature ka gain sabse zyada ho, wahi us node par split ke liye select hota hai.



Isme dikhaya gaya hai: Play Tennis dataset aur Outlook par split (9Y|5N → Sunny, Overcast, Rain), Pure vs Impure Split, Entropy aur Gini ke formulas, 6Yes|3No wala example, aur Entropy/Gini ka graph.

Original Handwritten Notes — Decision Tree, Page 3



Isme dikhaya gaya hai: H(C1) = 1 (impure) aur H(C2) = 0 (pure), Gini Impurity ka calculation (0.5 aur 0), features f1, f2, f3 se alag-alag decision trees, aur Information Gain ka formula.

6. Information Gain — Worked Example

Teacher ka example: root node me **14 samples = 9Y | 5N**. Feature **f_1** par split karne se do child nodes bante hain: **C_1 = 8 samples (6Y | 2N)** aur **C_2 = 6 samples (3Y | 3N)** — C_2 impure split hai. Ab hum f_1 ka gain nikalte hain.

Step 1 — Root Ki Entropy H(S)

$$H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.94$$

Step 2 — Dono Children Ki Entropy

$$H(C_1) = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8) \approx 0.81$$
$$H(C_2) = (3Y \mid 3N) = 1$$

Step 3 — Gain Nikalo

$$Gain(S, f_1) = 0.94 - [\ (8/14) \times 0.81 + (6/14) \times 1\ ]$$
$$= 0.94 - (0.463 + 0.429) = 0.94 - 0.892 \approx 0.049$$

(Calculator se exact value 0.0481 aati hai; 0.94 aur 0.81 round karne se teacher ko 0.049 mila hai. Dono approximately same hain.)

Step 4 — Doosre Feature Se Compare Karo

Isi tarah feature f_2 par split karke gain nikala, jo teacher ke notes me **0.051** aaya hai. Dono ko compare karo:

Feature	Information Gain	Result
f_1	0.049	—
f_2	0.051	**Zyada gain → f_2 ko root node banao**

> **Dhyaan dein:** Yahan f_1, f_2 generic (sample) features hain, Play Tennis table ke exact columns nahi. f_2 ke split ki poori ginti notes me nahi likhi, sirf uska result (0.051) diya hai. Teacher ne "Information Gain is basically calculated" likha hai, matlab har feature ka gain nikal kar compare kiya jaata hai.

7. Entropy vs Gini Impurity

Dono ke formulas dobara dekhte hain:

$$\textbf{Entropy: } H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$
$$\textbf{Gini: } G.I = 1 - \Sigma\,(p_i)^2$$

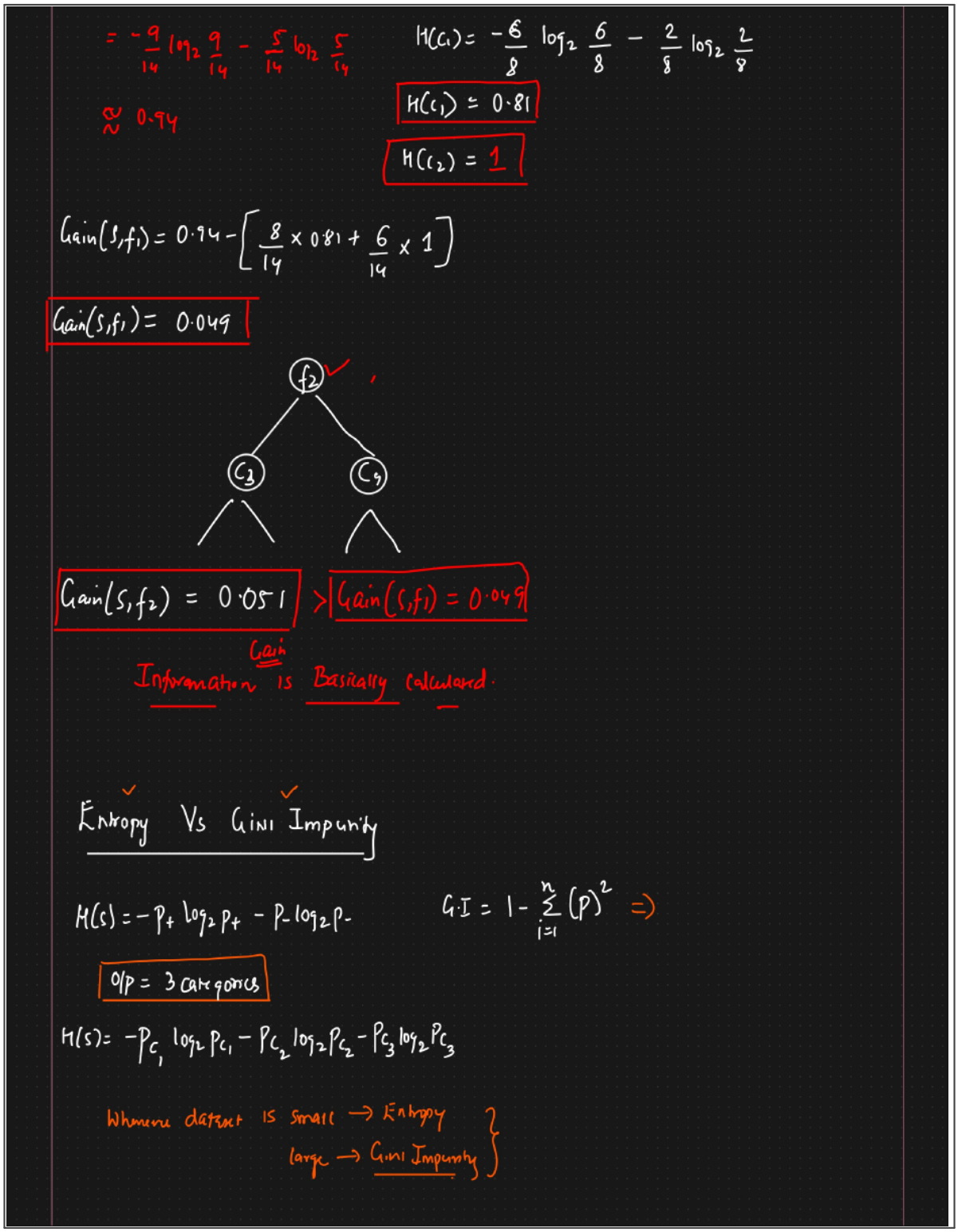
Agar output me 3 (ya zyada) categories ho, to entropy me bas ek aur term judta hai (teacher ne c_1, c_2, c_3 likh kar dikhaya hai):

$$H(S) = -P_{c1} \log_2 P_{c1} - P_{c2} \log_2 P_{c2} - P_{c3} \log_2 P_{c3}$$

Kab konsa use karein? Teacher ke notes ka rule: **dataset chhota ho → Entropy**, **dataset bada ho → Gini Impurity**. Wajah (extra): entropy me **log** nikalna padta hai jo square nikalne se slow hota hai. Chhote data me ye farak mehsoos nahi hota, lekin bade data me Gini fast hota hai.

Point	Entropy	Gini Impurity
Formula	$-\Sigma\ p \log_2 p$	$1 - \Sigma\ p^2$
Range (binary)	0 se 1	0 se 0.5
Speed	Thoda slow (log)	Fast (sirf square)
Kab use karein	Dataset chhota ho	Dataset bada ho
Algorithm	ID3 (Information Gain ke saath)	CART
sklearn criterion	"entropy"	"gini" (default)

Original Handwritten Notes — Decision Tree, Page 4



Isme dikhaya gaya hai: H(S) ≈ 0.94, H(C1) = 0.81, H(C2) = 1, Gain(S, f1) = 0.049, Gain(S, f2) = 0.051 (f2 select hua), aur Entropy vs Gini Impurity (3 categories ka formula, chhota dataset → Entropy, bada → Gini).

8. Numerical Features Par Decision Tree Split

Ab tak hamare features **categorical** the (jaise Outlook: Sunny / Overcast / Rain), isliye har category ke liye ek branch ban jaati thi. Lekin agar feature **numerical (continuous)** ho — jaise 2.3, 3.6, 4, 5.2 ... — to har alag value ki alag branch banana possible nahi (values hazaaron ho sakti hain). Isliye numerical feature par ek **threshold** chunte hain aur sawal poochte hain **"feature ≤ threshold?"**. Jawab **Yes** ya **No** — yaani hamesha 2 branches. Poori problem ye hai ki **best threshold kaunsa hai?**

8.1 Teacher Ka Dataset

Ek numerical feature $\mathbf{f_1}$ aur output (O/P) Yes/No. Total 7 samples, yaani root node = **4 Yes | 3 No**:

f1	2.3	3.6	4	5.2	6.7	8.9	10.5
O/P	Yes	Yes	No	No	Yes	No	Yes

8.2 Steps

- **Step 1 — Feature ki values ko sort karo** (chhote se bade), saath me unka output bhi. (Upar table already sorted hai.)
- **Step 2 — Har value ko ek candidate threshold maano.** Us threshold par data ko do hisso me todo: **≤ threshold** wale (Yes branch) aur **> threshold** wale (No branch).
- **Step 3 — Har threshold ka Information Gain nikalo** (section 5 wala wahi formula).
- **Step 4 — Jis threshold ka gain sabse zyada ho**, wahi us node ka split condition ban jaata hai.

8.3 Do Thresholds Ka Comparison (Notes Ka Example)

Threshold	Yes branch (≤ threshold)	No branch (> threshold)
1) $f_1 \le 2.3$	1Y | 0N (sirf 2.3)	3Y | 3N (baaki 6 samples)
2) $f_1 \le 3.6$	2Y | 0N (2.3 aur 3.6)	2Y | 3N (baaki 5 samples)

Ab dono ka Information Gain nikalte hain. Pehle root ki entropy:

$$\mathbf{H(S) = -(4/7)\log_2(4/7) - (3/7)\log_2(3/7) \approx 0.985}$$

Threshold 1 (≤ 2.3): H(1Y|0N) = 0 (pure) aur H(3Y|3N) = 1.

$$\mathbf{Gain = 0.985 - [\ (1/7) \times 0 + (6/7) \times 1\ ] = 0.985 - 0.857 \approx 0.128}$$

Threshold 2 (≤ 3.6): H(2Y|0N) = 0 (pure) aur H(2Y|3N) ≈ 0.971.

$$\mathbf{Gain = 0.985 - [\ (2/7) \times 0 + (5/7) \times 0.971\ ] = 0.985 - 0.694 \approx 0.29}$$

Threshold	Information Gain	Result
≤ 2.3	≈ 0.128	—
≤ 3.6	≈ 0.29	**Zyada gain → ye threshold select hua** (notes me tick)

> **Yaad rakhein:** Numerical feature par split = **threshold chuno → data do hisso me todo → Information Gain nikalo → sabse bada gain wala threshold rakho**. Is example me "$f_1 \leq 3.6$" jeeta: iski Yes branch pure ho gayi (2Y | 0N, leaf), aur No branch (2Y | 3N) impure hai, to us par split dobara chalega.

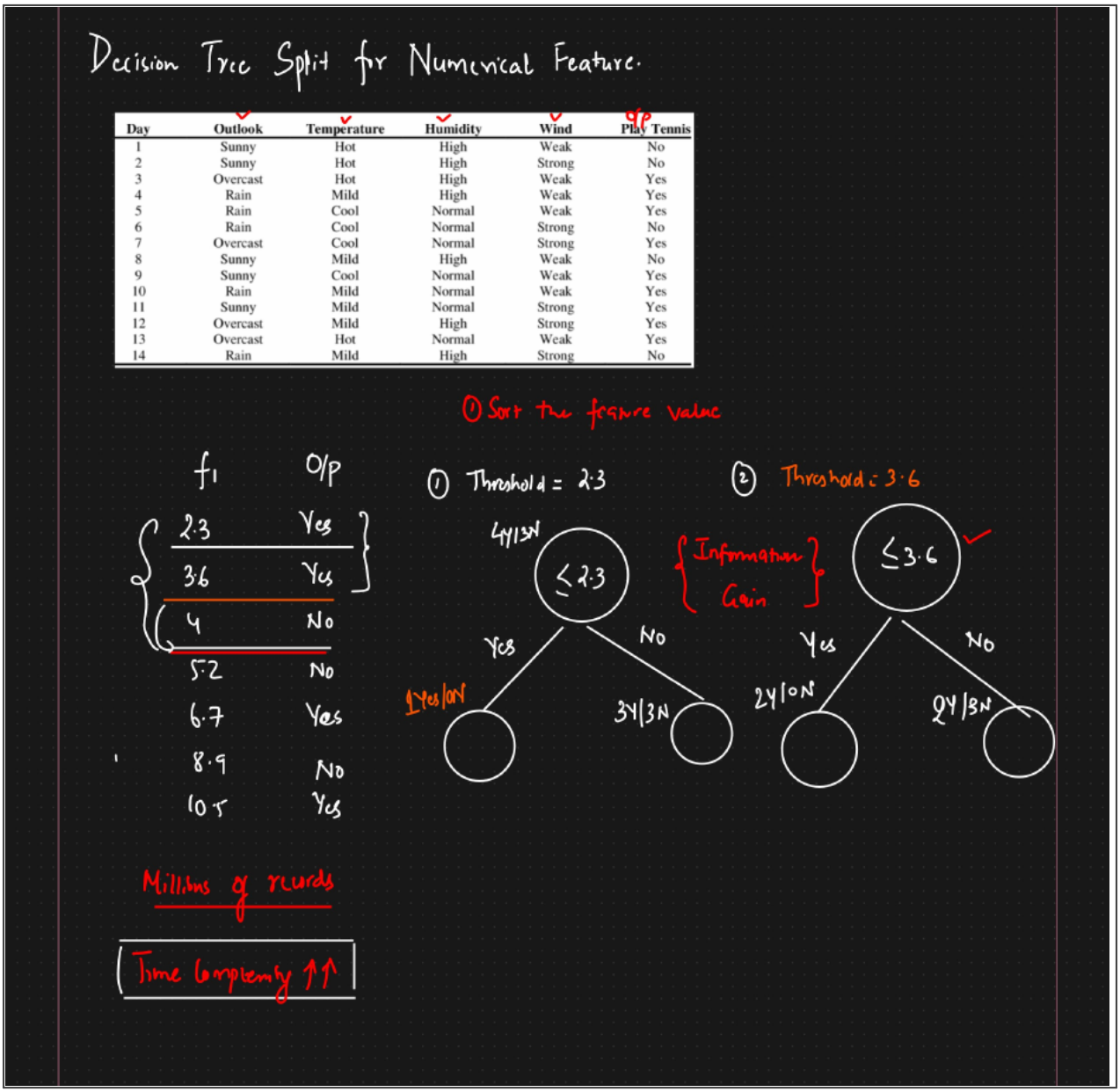
8.4 Time Complexity Ki Problem

Teacher ne last me likha hai: **Millions of records → Time Complexity ↑↑**. Wajah: har unique value ek possible threshold hai, aur har threshold ke liye Information Gain nikalna padta hai. Agar ek feature me millions of alag values hon, to millions thresholds check karne padenge — wo bhi har feature ke liye aur tree ke har node par. Isliye numerical features par tree banana slow ho jaata hai.

Extra (practical tips, notes me nahi):

- Sort karne ke baad ek hi pass me running Yes/No count update karke saare thresholds ke gain nikale ja sakte hain, har threshold ke liye data dobara scan nahi karna padta.
- Sirf wahi jagah threshold check karte hain jahan output label badalta hai (jaise 3.6 → 4 par Yes se No). Baaki jagah gain badhne ki umeed kam hoti hai.
- Scikit-learn me thresholds aam taur par do consecutive sorted values ka **midpoint** hote hain (jaise (3.6 + 4) / 2 = 3.8). Split wahi hota hai, bas threshold ki value thodi alag likhi jaati hai.
- Bahut bade data me values ko bins (groups) me baant kar kam thresholds check kiye jaate hain (histogram-based trees).

Original Handwritten Notes — Decision Tree Split (Numerical Feature)



Isme dikhaya gaya hai: Play Tennis dataset, numerical feature f1 ki values ko sort karna, do thresholds (2.3 aur 3.6) ke split (1Y|0N, 3Y|3N aur 2Y|0N, 2Y|3N), Information Gain se threshold 3.6 ka select hona, aur "Millions of records → Time Complexity" ki problem.

9. Bonus — Play Tennis Me Root Node Outlook Kyun Bana?

(Ye section teacher ke notes me nahi hai, lekin upar ke formulas ko asli dataset par lagakar dekhne ke liye bahut kaam ka hai.)

Root par 4 features me se kaun best hai, ye jaanne ke liye har feature ka Information Gain nikalte hain. Root entropy H(S) = 0.940. Outlook ke liye step-by-step:

Outlook value	Yes | No	Entropy	Weight
Sunny	2 | 3	0.971	5/14
Overcast	4 | 0	0	4/14
Rain	3 | 2	0.971	5/14

Gain(S, Outlook) = 0.940 − [(5/14)(0.971) + (4/14)(0) + (5/14)(0.971)] = 0.940 − 0.694 ≈ 0.247

Baaki features ka gain isi tareeke se nikalne par:

Feature	Information Gain	Result
Outlook	≈ 0.247	**Sabse zyada → root node**
Humidity	≈ 0.152	
Wind	≈ 0.048	
Temperature	≈ 0.029	

Isliye root node **Outlook** bana. Uske baad **Overcast** branch pure hai (4Y | 0N), to wo leaf ban gayi. **Sunny** aur **Rain** impure hain, to in dono par wahi process dobara chalta hai (bache hue features me se best gain wala chuno) — jab tak saare nodes pure na ho jaayein ya tree ki depth limit na aa jaaye.

Ye tree kitna gehra hone dein aur overfitting se kaise bachein — iske liye agla section (Pruning) dekho.

10. Bonus — Overfitting Aur Pruning (Pre-Pruning vs Post-Pruning)

(Ye section teacher ke notes me nahi hai, maine alag se joda hai. Decision Tree me ye topic exam aur interview dono me bahut puchha jaata hai.)

10.1 Problem — Tree Overfit Kyun Ho Jaata Hai?

Decision Tree ko agar bina rok-tok badhne dein, to wo tab tak split karta rehta hai jab tak **har leaf pure** na ho jaaye (jaise 4Y | 0N). Iska matlab tree itna gehra ho jaata hai ki kai leaf me sirf **1-2 samples** bachte hain. Aise tree ne training data ko, noise samet, ratt liya hota hai:

- **Training accuracy** bahut zyada (lagbhag 100%) → Low Bias
- **Test accuracy** gir jaati hai → High Variance

Isi ko **Overfitting** kehte hain (wahi concept jo Ridge/Lasso ke notes me tha). Iska ilaaj hai **Pruning** — yaani tree ko chhota karna, jaise bagiche me extra daaliyan kaat di jaati hain. Pruning ke do tareeke hain: **Pre-Pruning** aur **Post-Pruning**.

10.2 Pre-Pruning (Early Stopping)

Idea: tree **banate waqt hi** rok do — jaise hi koi condition poori ho jaaye, us node ko aur mat todo, use leaf bana do. Ye conditions hum pehle se **hyperparameters** ke roop me set karte hain:

Hyperparameter (sklearn)	Matlab
max_depth	Tree ki maximum gehraai (kitne levels tak split hoga)
min_samples_split	Kisi node ko todne ke liye kam se kam kitne samples chahiye
min_samples_leaf	Har leaf me kam se kam kitne samples hone chahiye
max_leaf_nodes	Poore tree me leaf nodes ki maximum sankhya
min_impurity_decrease	Split tabhi karo jab impurity (entropy/gini) itni kam ho

Fayda: simple aur fast, kyunki poora bada tree banana hi nahi padta. **Nuksaan:** kabhi-kabhi ek split abhi bekaar lagta hai lekin uske baad ke splits bahut achhe hote hain; early stopping se wo miss ho sakte hain, aur tree **underfit** ho sakta hai (bahut simple ban jaata hai).

10.3 Post-Pruning

Idea: pehle tree ko **poora (full depth tak)** banao, jo overfit hoga. Phir **neeche se upar** ek-ek branch ko check karo: agar kisi sub-tree ko kaat kar uski jagah ek **single leaf** (majority class wala) rakhne se validation/test performance kharab nahi hoti, to us sub-tree ko kaat do.

Chhota example: maan lo ek sub-tree ka node 4Y | 1N hai aur wo aage 3Y | 0N aur 1Y | 1N me tootta hai. Dusri wali branch me sirf 2 samples hain, jo noise ho sakte hain. Is sub-tree ko hata kar poore node ko ek leaf bana do jiska answer **Yes** (majority) ho. Agar validation accuracy kam nahi hui, to ye pruning sahi thi aur tree chhota aur simple ho gaya.

Popular method — Cost Complexity Pruning: tree ko ek combined cost se judge karte hain:

Cost = Tree ki error + α × (leaf nodes ki sankhya)

Yahan **α (alpha)** ek hyperparameter hai (scikit-learn me **ccp_alpha**). α = 0 par koi pruning nahi (poora tree), aur α jitna badhta hai utna zyada pruning hota hai aur tree utna chhota banta hai. Ye bilkul Ridge/Lasso ke λ jaisa idea hai — complexity par penalty.

Fayda: aam taur par better accuracy, kyunki pehle poore tree ko dekh liya jaata hai. **Nuksaan:** slow, kyunki pehle bada tree banana padta hai.

10.4 Comparison

Point	Pre-Pruning	Post-Pruning
Kab hota hai	Tree banate waqt	Tree poora banne ke baad
Kaise	Pehle se limits set karo (max_depth, min_samples_leaf...)	Bekaar branches kaat kar leaf se replace karo
Speed	Fast	Slow (pehle poora tree banta hai)
Risk	Underfitting (jaldi ruk gaye)	Kam; aam taur par zyada accurate
sklearn me	max_depth, min_samples_split, min_samples_leaf	ccp_alpha

Scikit-learn me dono aise likhte hain:

```
# Pre-pruning
DecisionTreeClassifier(max_depth=3, min_samples_leaf=5)

# Post-pruning (cost complexity)
DecisionTreeClassifier(ccp_alpha=0.01)
```

Yaad rakhein: Practice me in hyperparameters (max_depth, ccp_alpha etc.) ki best value **cross-validation** (jaise GridSearchCV) se chuni jaati hai. Pre-pruning = **"banate waqt roko"**, Post-pruning = **"bana ke kaato"**.

11. Poore Notes Ka Quick Revision

Topic	Yaad rakhne wali baat
Decision Tree	Nested if-else jaisa tree: root → decision nodes → leaf nodes
Types	ID3 (Entropy + Info Gain), CART (Gini, binary splits)
Pure / Impure split	Pure = node me ek hi class (leaf ban jaata hai); Impure = mixed, aur split karo
Entropy	$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$; pure = 0, 50-50 = 1
Gini Impurity	$G.I = 1 - \Sigma p^2$; pure = 0, 50-50 = 0.5
Information Gain	$Gain(S, f) = H(S) - \Sigma (\lvert S_v \rvert / \lvert S \rvert) \times H(S_v)$; sabse bada gain wala feature chuno
Worked example	$Gain(S, f_1) = 0.049 < Gain(S, f_2) = 0.051 \rightarrow f_2$ root
Entropy vs Gini	Dataset chhota → Entropy, bada → Gini
Numerical feature	Sort karo → threshold (feature ≤ t) try karo → sabse bada Information Gain wala chuno (jaise ≤ 3.6)
Overfitting	Tree bahut gehra ho jaaye to training data ratt leta hai (high variance); ilaaj = pruning
Pre-pruning	Tree banate waqt roko: max_depth, min_samples_split, min_samples_leaf
Post-pruning	Poora tree banao, phir bekaar branches kaato (ccp_alpha)
Numerical ki problem	Millions of records → bahut saare thresholds → time complexity zyada